Authors

> **R4176c Computer Vision**
>
> Бахорина Ольга Александровна, ИСУ 506086

The task and guidelines were prepared by Andrei Zhdanov and Sergei Shavetov, ITMO University, 2025.

Practical Assignment No 2. Hough Transform

Study of the Hough transform to find geometric primitives.

To implement the current practical assignment task we would need OpenCV, scikit-image and NumPy libraries along with image display functions we wrote during the Image Processing class. We import OpenCV's `cv2` library as `cv` for easier use.

In [1]:
```
# Import OpenCV library as both cv2 and cv
import cv2
import cv2 as cv
# Import NumPy library as np
import numpy as np
# Import math for, yes we just need math
import math
# Import scikit-image transform library for Hough transformation
import skimage.transform
# Import functions from our utility library
from pa_utils import ShowImages, exit
```

Introduction

In the current practical assignment, we will learn some basic object detection methods that allow searching for objects on an image using the common locus method.

Common Locus of Points

The main principle of the Hough transform [1] is to find a common *locus of points*. For example, this approach is used when designing a triangle along three given sides. At first one side of the triangle is laid off, after that the ends of the segment are considered as the centers of circles with radii equal to the lengths of the second and third segments. The intersection of the two circles is the common locus of points, from where the

segments are drawn to the ends of the first segment. In other words, a *voting* of two points was held in favor of the probable location of the third vertex of the triangle. As a result of *voting* the *winner* was the point that got two *votes* (the points on the circles got one vote each, and outside them, they got zero), see Fig. 1.

[1]: Hough, P. V. C. (1962, December 18). Method and means for recognizing complex patterns (U.S. Patent No. 3,069,654). Google Patents.

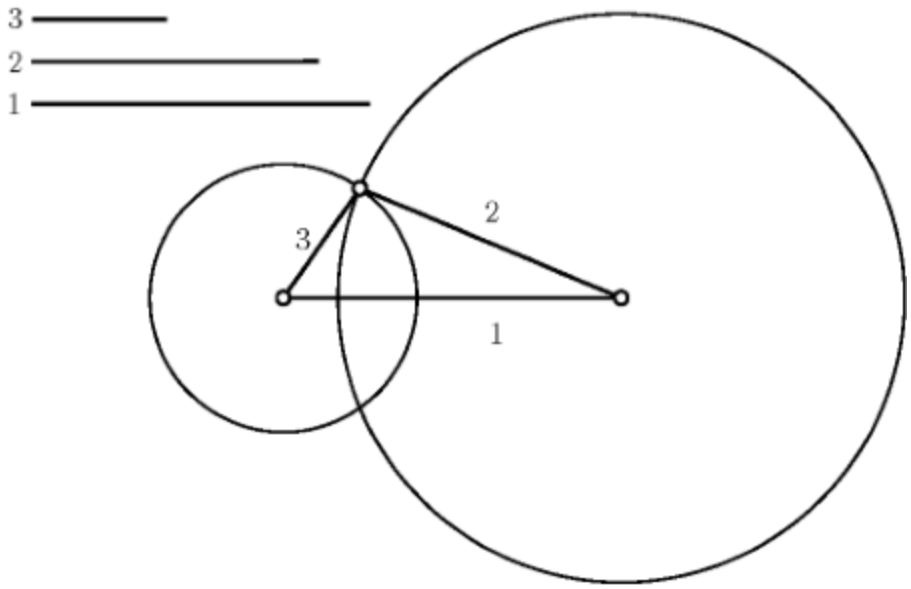


Fig. 1. *Finding a triangle by the given three sides.*

Let's generalize this idea for working with real data when the image has a large number of special feature points participating in the vote. Let us assume that it is necessary to search a circle of known radius in a binary point set, and in this set, there may also be false points that do not lie on the desired circle. The set of possible circle centers for the desired radius around each characteristic point forms a circle of radius , see Fig. 2 below. Thus, the point corresponding to the maximum intersection of the number of circles will be the center of the required radius circle.

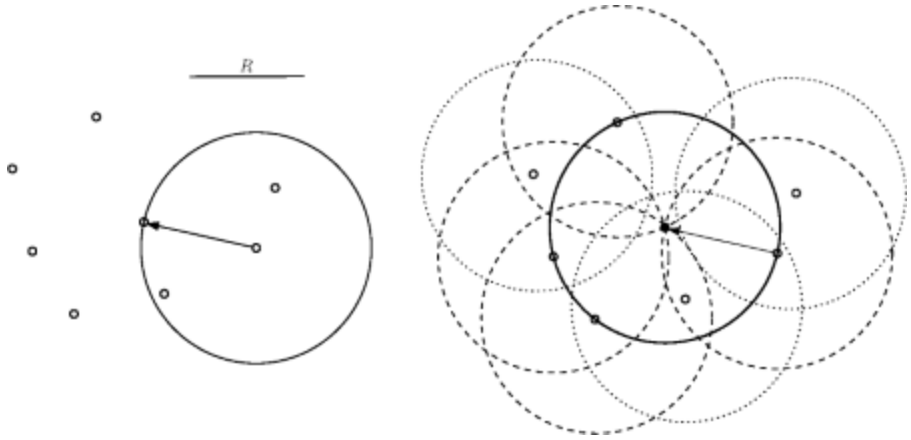


Fig. 2. *Searching a circle of known radius in a point set.*

Classic Hough Transform

The classic Hough transform is based on the considered point voting idea. It was originally designed to select lines on binary images. The Hough transform uses the parameter space to search for geometric primitives. The most well-known parametric equation of lines is:

This equation has a problem as it can not be used to define lines that are parallel to the axis, so the Hough transform uses another common parametric equation of a line by a point and an inclination angle:

where

- is the radius vector drawn from the origin to the line;
- is the inclination angle of the radius vector.

Let the straight line in the Cartesian coordinate system be given by the given equation, from which it is easy to calculate the radius vector and angle . Then we can define the Hough parameter space with coordinated and . As a result, each line of the source Cartesian space will be represented by a single point with coordinates $_0$ $_0$ in the Hough parameter space, see Fig. 3.

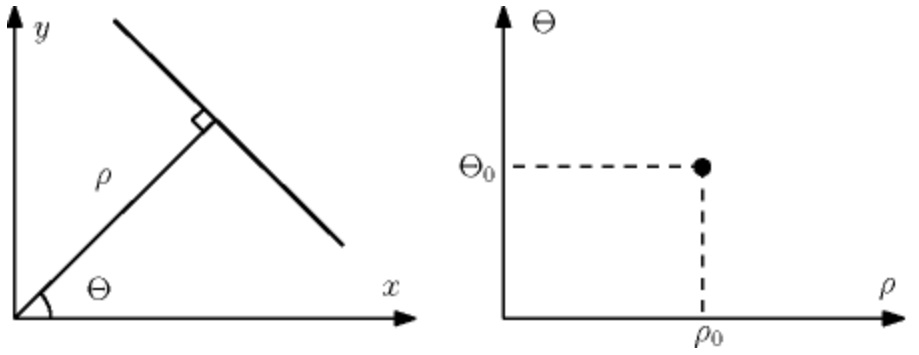


Fig. 3. *Representation of a straight line in Hough space.*

During the Hough transform all lines that pass via points in the source Cartesian space are converted to points in the Hough parameter space and *vote* for corresponding lines. As a result, the parameter space sums upvotes given for various lines. In general, the parameter space is continuous, however, in a digital form, it is stored as a matrix. Therefore, in the Hough transform the Hough space is called *accumulator* and is a matrix that stores voting information.

An infinite number of straight lines can be drawn passing through any of the points in the source Cartesian coordinate space and they will generate a sinusoidal response function

in the parameter space. Thus, any two sinusoidal response functions in the parameter space will intersect at the point only if the points originating them in the source Cartesian space lie on a single straight line, see Fig. 4. Based on this, we can conclude that to find straight lines in the source space, it is necessary to find all the local maxima of the accumulator matrix.

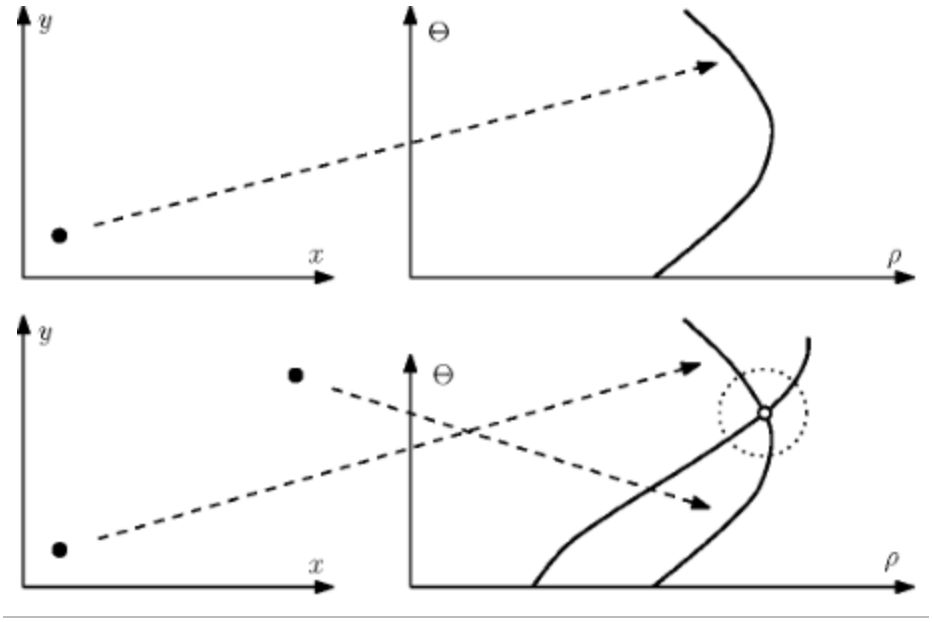


Fig. 4. *Voting in the Hough parameter space.*

The considered line search algorithm can be used in the same way to search for any other curve described in space by some function with a certain number of parameters 1 2 n , which will only affect the dimension of the parameter space.

Let us use the Hough transform to search for circles of a given radius . It is known that a circle on a plane is described by the formula:

$$_0{}^2 \quad _0{}^2 \quad ^2$$

The set of centers of all possible circles of radius passing through a feature point forms a circle of radius around that point. Due to this the response function in the Hough transform for finding circles is a circle of the same size centered at the voting point, see Fig. 2. Then, similarly to the previous case, it is necessary to find the local maxima of the accumulator function in the space of parameters , which will be the centers of the required circles.

The Hough transform is invariant to shift, scaling, and rotation. Taking into account that under projective transformations of three-dimensional space, straight lines always go only to straight lines (in the degenerate case, to points), the Hough transform makes it possible to detect lines invariably not only to affine transformations of the plane but also to the group of projective transformations in space.

Task 1. Search for Lines

Take three arbitrary images containing lines. Search for straight lines using various implementations of the Hough transform. Plot the found lines on the original image. Mark the start and end points of the lines. Determine the lengths of the shortest and longest lines, and calculate the number of lines found.

1.1 Hough Transform for Lines with OpenCV

OpenCV library provides two implementations for the Hough transform algorithm for search of the straight lines:

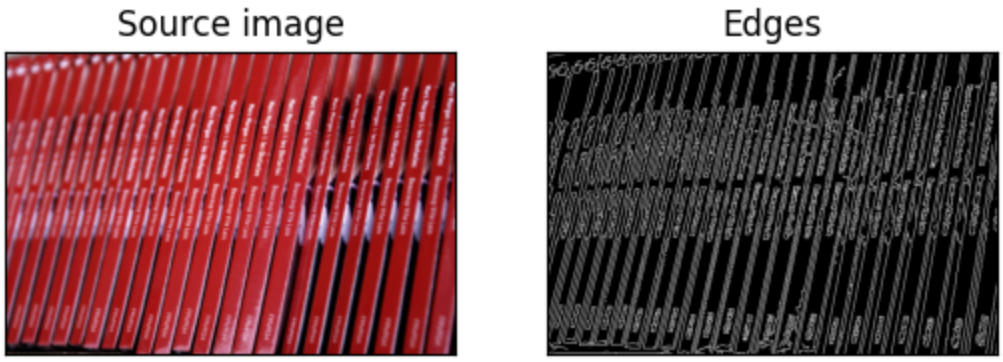
- `cv2.HoughLines(image, rho, theta, threshold) -> lines` function performs the classic Hough line transform of an `image` and searches for straight lines. The `rho` and `theta` parameters define the subdivision of Hough parameter space for the corresponding axis. The `threshold` parameter defines the threshold, i.e., the number of votes that a line should get to be added to the returned `lines` array. Each line in returned `lines` array is an infinite line which is defined by `rho` ( ) and `theta` ( ) parameters.
- `cv2.HoughLinesP(image, rho, theta, threshold, minLineLength, maxLineGap) -> lines` function performs the probabilistic Hough line transform of an `image` and searches for straight lines. The main function parameters are the same, however, two extra are added: these are `minLineLength` for a minimum line length, so no line shorter than this won't be selected, and `maxLineGap` for a maximum gap between two segments of the line to consider them to be the same line instead of two separate ones. The returned 2-dimensional `lines` array contains the start and end points of each of the found line segments as arrays of four: 1 1 2 2 .

1.1.1 Classic Hough Transform for Lines with OpenCV

To execute the Hough line transform in OpenCV first have to preprocess an image to get edges by executing the Canny algorithm.

In [2]:
```
# Read an image from a file in BGR
fn = "images/barcode.png"
I1 = cv.imread(fn, cv.IMREAD_COLOR)
if not isinstance(I1, np.ndarray) or I1.data == None:
  print("Error reading file \"{}\"".format(fn))
  exit()

# Preprocess with Canny algorithm
I1edge = cv.Canny(I1, 50, 200, None, 3)
# Display it
```

```
ShowImages([("Source image", I1),
            ("Edges", I1edge)], 2)
```



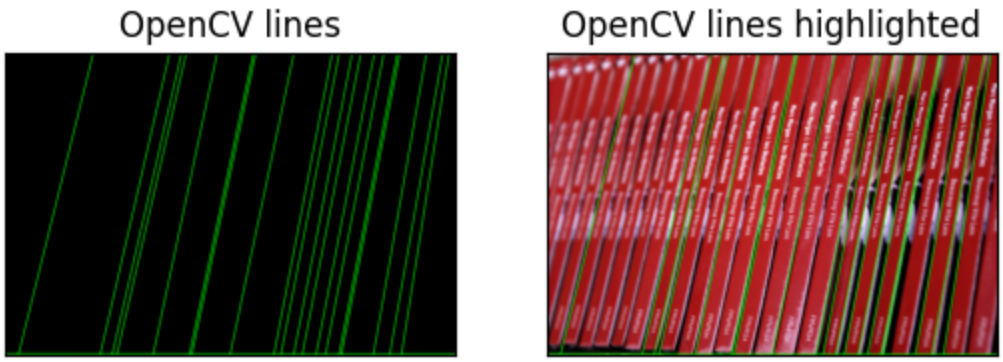
Then run the Hough line transform to get line parameters.

In [3]:
```
# Find lines with Hough transform
# Distance resolution: 1 pix
# Angle resolution: 1 deg in rad
# Accumulator threshold: 200
I1classic = cv.HoughLines(I1edge, 1, np.pi / 180, 200)
```

Finally, we can draw infinite lines by returned parameters with the help of the `cv2.line()` function.

In [4]:
```
# Create output images
I1classic_out = I1.copy()
I1classic_lines = np.zeros_like(I1)
h, w = I1.shape[:2]
l = math.sqrt(h ** 2 + w ** 2)

# Go through all found lines and display them
if I1classic is not None:
  for i in range(0, len(I1classic)):
    rho = I1classic[i][0][0]
    theta = I1classic[i][0][1]
    a, b = math.cos(theta), math.sin(theta)
    x0, y0 = a * rho, b * rho
    pt1 = np.int32((x0 - l * b, y0 + l * a))
    pt2 = np.int32((x0 + l * b, y0 - l * a))
    cv.line(I1classic_out, pt1, pt2, (0, 255, 0), 1, cv.LINE_AA)
    cv.line(I1classic_lines, pt1, pt2, (0, 255, 0), 1, cv.LINE_AA)

# Display it
print("Found {} lines".format(len(I1classic)))
ShowImages([("OpenCV lines", I1classic_lines),
            ("OpenCV lines highlighted", I1classic_out)], 2)
```

```
Found 20 lines
```



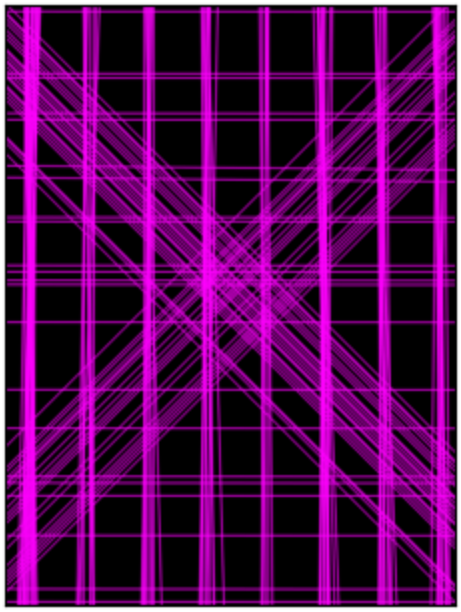
1.1.2 Self-work

> ***Self-work***
>
> Take **three** arbitrary images and execute the classic Hough transform with OpenCV. Display the found lines and print their number.
>
> ***Notes.***
>
> 1. *You may have to change the Hough transform parameters to match your images.*

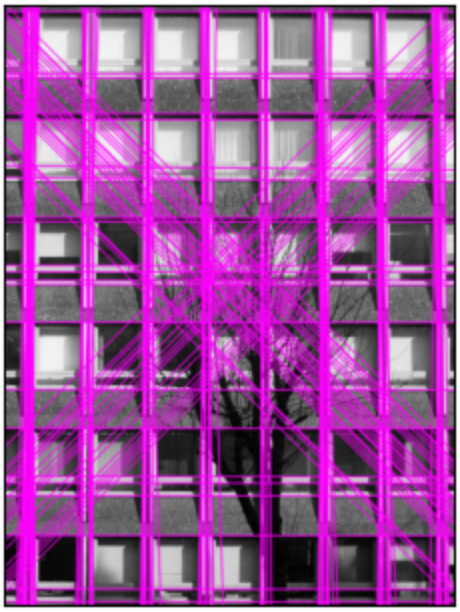
In [5]:
```
line_sources = {
    "facade": cv.imread("images/lines1.jpg", cv.IMREAD_COLOR),
    "sign": cv.imread("images/lines2.jpg", cv.IMREAD_COLOR),
    "railing": cv.imread("images/lines3.jpg", cv.IMREAD_COLOR),
}

for name, img in line_sources.items():
  edges = cv.Canny(img, 60, 180)
  lines = cv.HoughLines(edges, 1, np.pi / 360, 200)
  diag = math.hypot(*img.shape[:2])

  canvas = img.copy()
  blank = np.zeros_like(img)
  if lines is not None:
    for rho, theta in lines[:, 0]:
      ct, st = math.cos(theta), math.sin(theta)
      x0, y0 = ct * rho, st * rho
      p1 = np.int32((x0 - diag * st, y0 + diag * ct))
      p2 = np.int32((x0 + diag * st, y0 - diag * ct))
      cv.line(canvas, p1, p2, (255, 0, 255), 1, cv.LINE_AA)
      cv.line(blank, p1, p2, (255, 0, 255), 1, cv.LINE_AA)

  print(f"{name}: {0 if lines is None else len(lines)} lines")
  ShowImages([(f"{name} lines", blank), (f"{name} overlay", canvas)], 2)
```

```
facade: 150 lines
```

facade lines

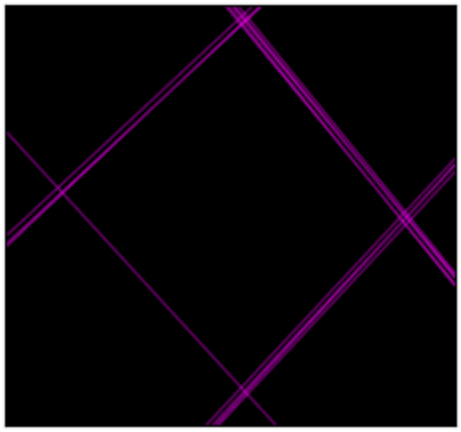


facade overlay



```
sign: 13 lines
```

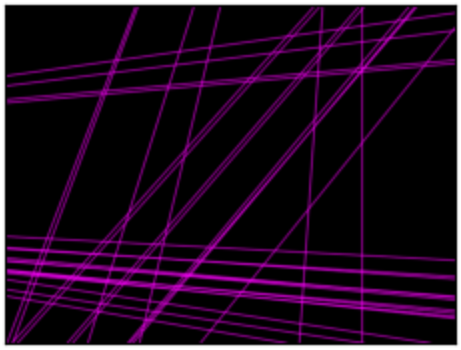
sign lines



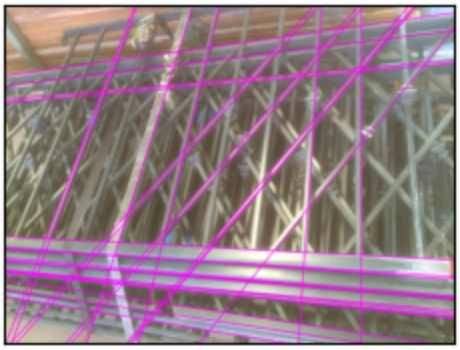
sign overlay


```
railing: 31 lines
```

railing lines



railing overlay



1.1.3 Probabilistic Hough Transform for Lines with OpenCV

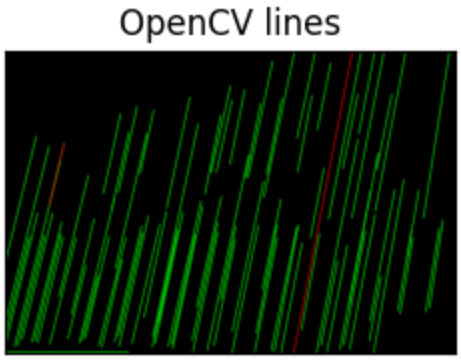
The probabilistic Hough line transform is executed similarly:

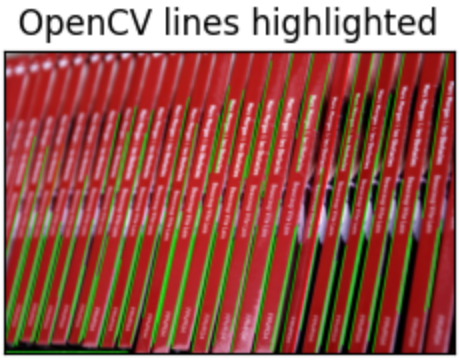
In [6]:
```
# Find lines with Probabilistic Hough transform
# Distance resolution: 1 pix
# Angle resolution: 1 deg in rad
# Accumulator threshold: 120
# Minimum line length: 80
# Maximum line gap: 4
I1prob = cv.HoughLinesP(I1edge, 1, np.pi / 180, 120, None, 80, 4)
```

The probabilistic Hough transform will return an array of pairs of the endpoints of the found line segments which we can use to draw them on top of the source image:

In [7]:
```
# Create and output image
I1prob_out = I1.copy()
I1prob_lines = np.zeros_like(I1)

# Go through all found lines and display them
if I1prob is not None:
  min_line = max_line = I1prob[0][0]
  min_line_len = max_line_len = math.sqrt((min_line[0] - min_line[2]) ** 2 +
  for i in range(1, len(I1prob)):
    l = I1prob[i][0]
    line_len = math.sqrt((l[0] - l[2]) ** 2 + (l[1] - l[3]) ** 2)
    if line_len < min_line_len:
      min_line_len = line_len
      min_line = l
    if line_len > max_line_len:
      max_line_len = line_len
      max_line = l
    cv.line(I1prob_out, (l[0], l[1]), (l[2], l[3]), (0, 255, 0), 1, cv.LINE_
    cv.line(I1prob_lines, (l[0], l[1]), (l[2], l[3]), (0, 255, 0), 1, cv.LIN

  # Draw min and max lines
  cv.line(I1prob_out, (min_line[0], min_line[1]), (min_line[2], min_line[3])
  cv.line(I1prob_lines, (min_line[0], min_line[1]), (min_line[2], min_line[3
  cv.line(I1prob_out, (max_line[0], max_line[1]), (max_line[2], max_line[3])
  cv.line(I1prob_lines, (max_line[0], max_line[1]), (max_line[2], max_line[3

  # Display it
  print("Found {} lines".format(len(I1prob)))
  print("The shortest found line length is {}".format(min_line_len))
  print("The shortest found line is ({}, {}) - ({}, {})".format(min_line[0],
  print("The longest found line length is {}".format(max_line_len))
  print("The longest found line is ({}, {}) - ({}, {})".format(max_line[0],
ShowImages([("OpenCV lines", I1prob_lines),
            ("OpenCV lines highlighted", I1prob_out)], 2)
```

```
Found 89 lines
The shortest found line length is 85.37564055396598
The shortest found line is (58, 207) - (78, 124)
The longest found line length is 408.5156055770697
The longest found line is (383, 401) - (461, 0)
```



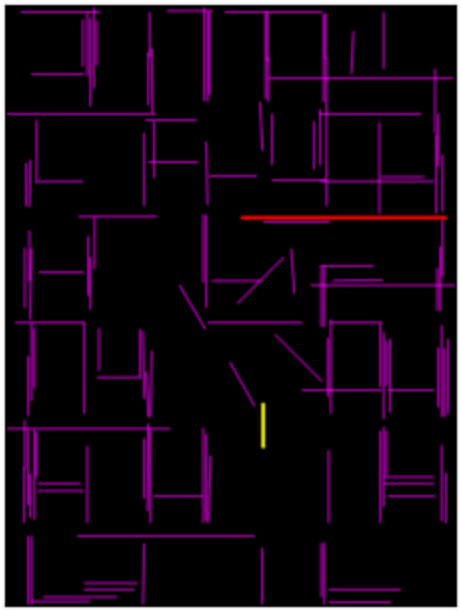


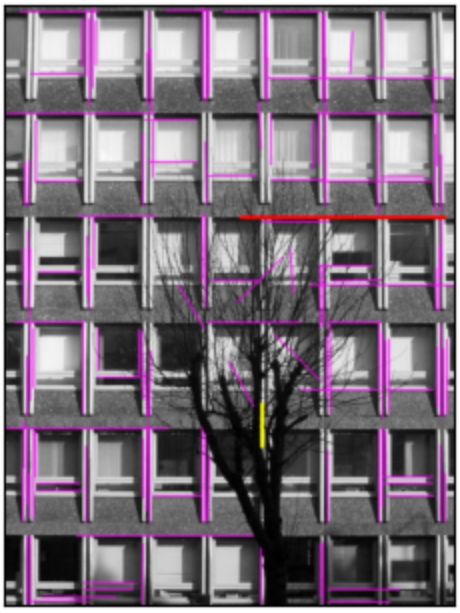
1.1.4 Self-work

> ***Self-work***
>
> Take **three** arbitrary images and execute the probabilistic Hough transform with OpenCV. Display the found lines, print their number, and print the shortest and the longest line parameters.
>
> ***Notes.***
>
> 1. *You may have to change the Hough transform parameters to match your images.*

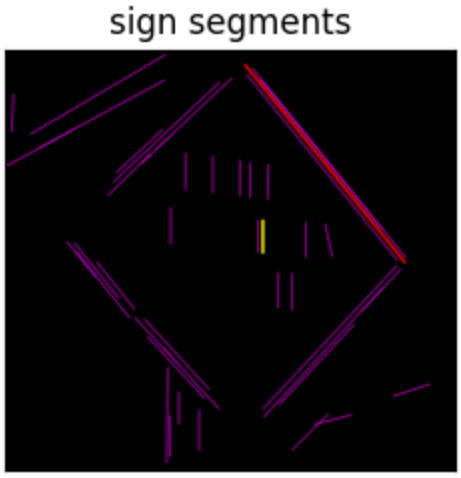
In [8]:
```
def seg_len(s):
  return math.hypot(s[2] - s[0], s[3] - s[1])

for name, img in line_sources.items():
  edges = cv.Canny(img, 60, 180)
  segments = cv.HoughLinesP(edges, 1, np.pi / 360, 60, None, 40, 3)

  canvas = img.copy()
  blank = np.zeros_like(img)
  if segments is not None:
    segs = segments[:, 0]
    lengths = [seg_len(s) for s in segs]
    short, long = int(np.argmin(lengths)), int(np.argmax(lengths))
    for s in segs:
      cv.line(canvas, (s[0], s[1]), (s[2], s[3]), (255, 0, 255), 1, cv.LINE_
      cv.line(blank, (s[0], s[1]), (s[2], s[3]), (255, 0, 255), 1, cv.LINE_A
    for layer in (canvas, blank):
      sh, lg = segs[short], segs[long]
      cv.line(layer, (sh[0], sh[1]), (sh[2], sh[3]), (0, 255, 255), 2, cv.LI
      cv.line(layer, (lg[0], lg[1]), (lg[2], lg[3]), (0, 0, 255), 2, cv.LINE

    print(f"{name}: {len(segs)} segments")
    print(f"  shortest {lengths[short]:.1f}px {tuple(segs[short])}")
    print(f"  longest {lengths[long]:.1f}px {tuple(segs[long])}")
  ShowImages([(f"{name} segments", blank), (f"{name} overlay", canvas)], 2)
```

```
facade: 139 segments
  shortest 40.0px (np.int32(240), np.int32(372), np.int32(240), np.int32(41
2))
  longest 190.0px (np.int32(220), np.int32(198), np.int32(410), np.int32(19
8))
```

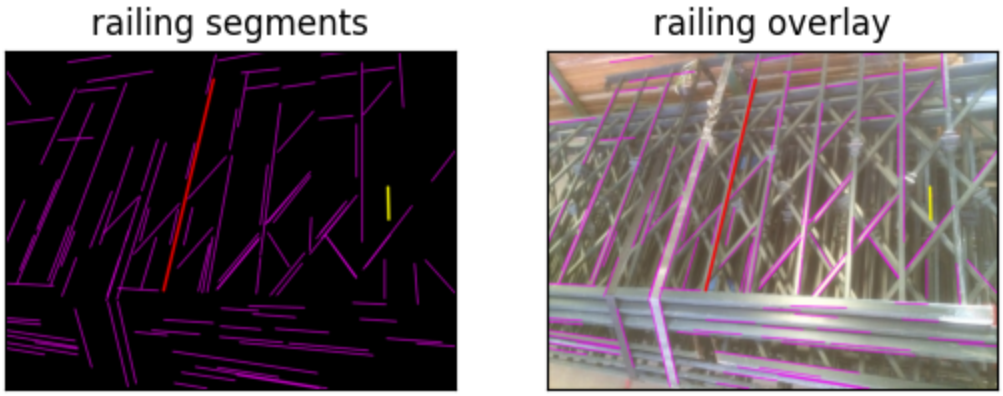





```
sign: 44 segments
  shortest 40.0px (np.int32(319), np.int32(251), np.int32(319), np.int32(21
1))
  longest 315.0px (np.int32(297), np.int32(18), np.int32(495), np.int32(26
3))
```




```
railing: 110 segments
  shortest 40.0px (np.int32(474), np.int32(170), np.int32(475), np.int32(21
0))
  longest 268.3px (np.int32(196), np.int32(298), np.int32(258), np.int32(3
7))
```



1.2 Hough Transform for Lines with scikit-image

Unfortunately, OpenCV does not provide the functionality to obtain the Hough accumulator matrix. However, the scikit-image library also provides functions that we can use to perform the Hough transform, and opposite to OpenCV, scikit-image executes it in several steps thus allowing us to process and display the Hough parameter space.

Same as OpenCV, scikit-image provides two implementations for the Hough transform algorithm for search of the straight lines. These are classic and probabilistic approaches. The main difference between OpenCV and scikit-image libraries is that in the classic approach the Hough transform contains two steps:

1. Calculation of the Hough parameter space;
2. Search for peaks in the Hough parameter space to find lines.

The following functions for Hough transform for lines are provided in scikit-image:

- `skimage.transform.hough_line(image, thetas) -> hspace, angles, distances` function performs the classic Hough line transform of an `image` and calculates the Hough accumulator matrix (parameter space). The `thetas` parameter defines an array of angles in radians to subdivide the Hough parameter space (defaults to 180 values in the range). It returns a tuple with three items, which are `hspace` with the Hough transform accumulator matrix; `angles` with the array of angles in radians which define the Hough parameter space; and `distances` with the array of distances which define the Hough parameter space.
- `skimage.transform.hough_line_peaks(hspace, angles, distances, min_distance, min_angle, threshold num_peaks) -> accum, theta, rho` function performs the search for peak values in the Hough parameter space acquired at the previous step. The input parameters list contains the data acquired from `skimage.transform.hough_line()` function with additional parameters for minimum distance and angle between two lies to consider them separate lines

(`min_distance` and `min_angle` parameters, defined in the accumulator matrix dimensions), the minimum peak value `threshold` to choose (defaults to the half of the maximum `hspace` value) and the maximum number of peaks to search for (`num_peaks` parameter). It returns a tuple of three items (`accum` , `theta` , `rho`) corresponding to peak values in the Hough accumulator matrix. Each line in the returned arrays is an infinite line which is defined by a pair of `rho` ( ) and `theta` ( ) parameters with the same array indices that scored the corresponding `accum` *votes*.

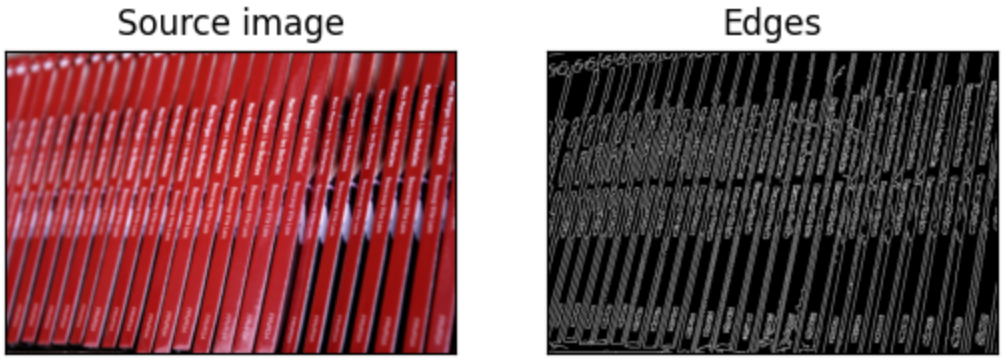
- `skimage.transform.probabilistic_hough_line(image, threshold, line_length, line_gap, theta) -> lines` function performs the probabilistic Hough line transform of an `image` and searches for straight lines. The main function parameters are similar to the ones used in OpenCV. They are the `threshold` for the minimum required peak value, the minimum line length to consider (`line_length` parameter), the maximum gap between two segments of the same line (`line_gap` parameter), and an array `theta` of angles to use during transformation. The returned 3-dimensional `lines` array contains the start and end points of each of the found line segments as 2D arrays with four elements:

  1 1 2 2 .

1.2.1 Classic Hough Transform for Lines with scikit-image

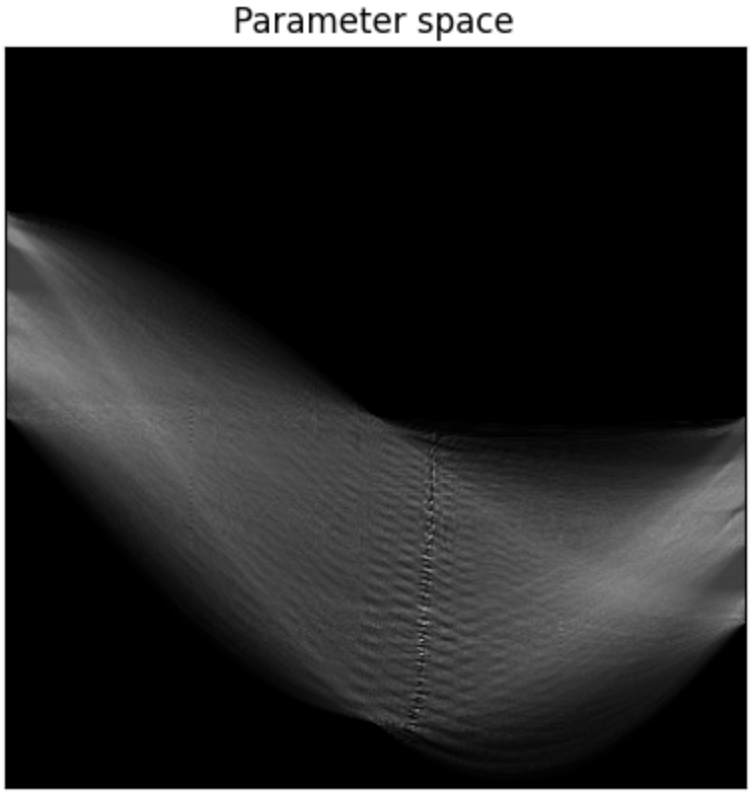
Same with the OpenCV library, to execute the Hough line transform in scikit-image first we have to preprocess an image to get edges by executing the Canny algorithm.

In [9]:
```
# Read an image from file in BGR
fn = "images/barcode.png"
I1 = cv.imread(fn, cv.IMREAD_COLOR)
if not isinstance(I1, np.ndarray) or I1.data == None:
  print("Error reading file \"{}\"".format(fn))
  exit()

# Preprocess with Canny algorithm
I1edge = cv.Canny(I1, 50, 200, None, 3)
# Display it
ShowImages([("Source image", I1),
            ("Edges", I1edge)], 2)
```



Then run the Hough line transform to get the Hough parameter space accumulator matrix. Since now we have the accumulator matrix we can display it as well to see how the parameter space is formed.

In [10]:
```
# Do Hough transform
# We will use 360 angles instead of the default 180
thetas = np.linspace(-np.pi / 2, np.pi / 2, 360, endpoint = False)
I1h, angles, distances = skimage.transform.hough_line(I1edge, theta = thetas
# Show Hough parameter space
# We will resize it to make square for a better display
ShowImages([("Parameter space", cv.resize(I1h.astype(np.float32) / np.max(I1
```



Now we can search for the maxima values in the Hough parameter space.

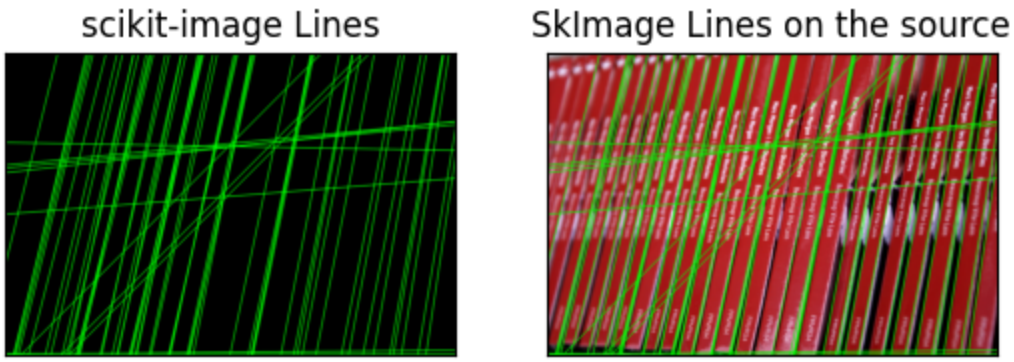
In [11]:
```
# Find lines with Hough transform
# Minimum distance between lines: 0
```

```
# Minimum angle between lines: 0
I1accum, I1theta, I1rho = skimage.transform.hough_line_peaks(I1h, angles, di
```

Finally, we can draw the infinite lines found by the scikit-image Hough transform implementation with subsequent calls to the `cv2.line()` function.

In [12]:
```
# Create output images
I1classic_out = I1.copy()
I1classic_lines = np.zeros_like(I1)
h, w = I1.shape[:2]
l = math.sqrt(h ** 2 + w ** 2)

# Go through all found lines and display them
if I1theta is not None and I1rho is not None:
  for i in range(0, len(I1theta)):
    a, b = math.cos(I1theta[i]), math.sin(I1theta[i])
    x0, y0 = a * I1rho[i], b * I1rho[i]
    pt1 = np.int32((x0 - l * b, y0 + l * a))
    pt2 = np.int32((x0 + l * b, y0 - l * a))
    cv.line(I1classic_out, pt1, pt2, (0, 255, 0), 1, cv.LINE_AA)
    cv.line(I1classic_lines, pt1, pt2, (0, 255, 0), 1, cv.LINE_AA)

# Display it
print("Found {} lines".format(len(I1theta)))
ShowImages([("scikit-image Lines", I1classic_lines),
            ("SkImage Lines on the source", I1classic_out)], 2)
```

```
Found 63 lines
```

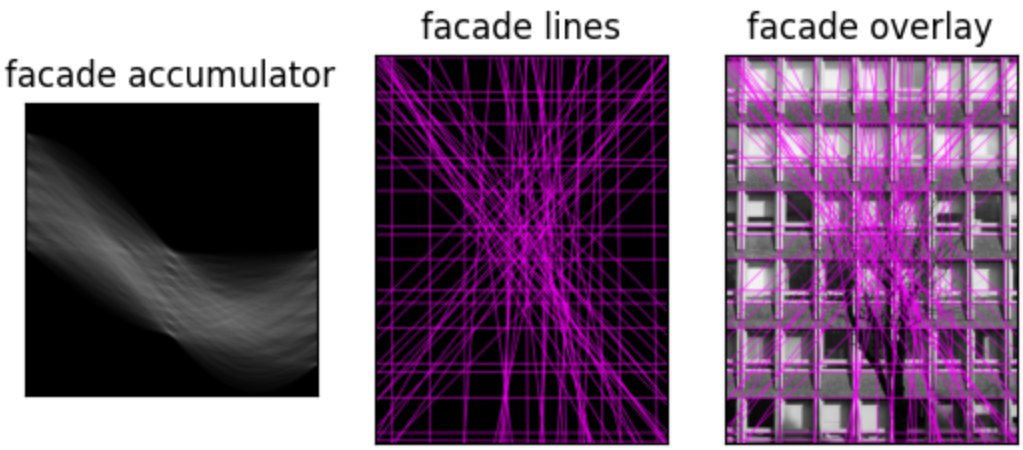


1.2.2 Self-work

> ***Self-work***
>
> Take **three** arbitrary images and execute the classic Hough transform with the scikit-image library. Display the Hough parameter space and found lines. Print the found lines their number.
>
> ***Notes.***
>
> 1. *You may have to change the Hough transform parameters to match your images.*

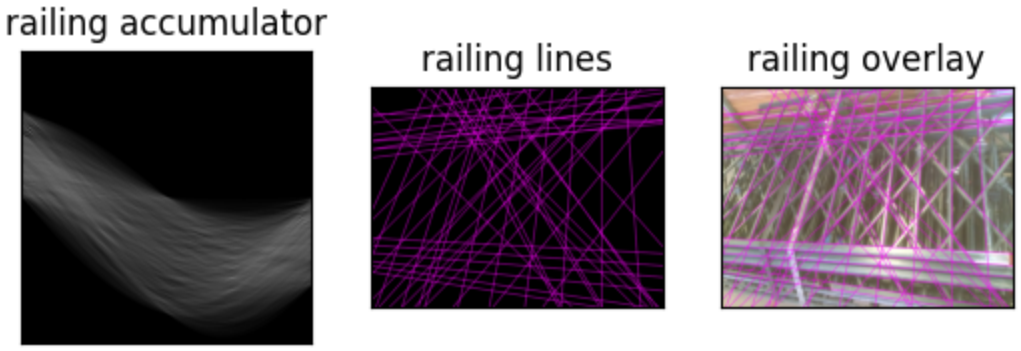
In [13]:
```
angle_grid = np.linspace(-np.pi / 2, np.pi / 2, 720, endpoint = False)

for name, img in line_sources.items():
  edges = cv.Canny(img, 60, 180)
  hspace, angles, dists = skimage.transform.hough_line(edges, theta = angle_
  _, peak_t, peak_r = skimage.transform.hough_line_peaks(hspace, angles, dis

  diag = math.hypot(*img.shape[:2])
  canvas = img.copy()
  blank = np.zeros_like(img)
  for t, r in zip(peak_t, peak_r):
    ct, st = math.cos(t), math.sin(t)
    x0, y0 = ct * r, st * r
    p1 = np.int32((x0 - diag * st, y0 + diag * ct))
    p2 = np.int32((x0 + diag * st, y0 - diag * ct))
    cv.line(canvas, p1, p2, (255, 0, 255), 1, cv.LINE_AA)
    cv.line(blank, p1, p2, (255, 0, 255), 1, cv.LINE_AA)

  space = cv.resize(hspace.astype(np.float32) / hspace.max(), (hspace.shape[
  print(f"{name}: {len(peak_t)} lines")
  ShowImages([(f"{name} accumulator", space), (f"{name} lines", blank), (f"{
```

```
facade: 77 lines
```




```
sign: 16 lines
```


```
railing: 50 lines
```



1.2.3 Probabilistic Hough Transform for Lines with scikit-image

The probabilistic Hough line transform with scikit-image is executed in a way similar to OpenCV. The only two differences are that now you have to use the `skimage.transform.probabilistic_hough_line(image, threshold, line_length, line_gap, theta) -> lines` function and returned lines parameters ara in 3D array instead of 2D array in OpenCV. So, the line length calculation will be as follows:

```
line = lines[i]
line_len = math.sqrt((line[0][0] - line[1][0]) ** 2 + (line[0][1] - line[1][1]) ** 2)
```

And the line drawing will be as follows:

```
color = (0, 0, 255) # Color in OpenCV is defined in BGR, so this will be the Red color
cv.line(Iout, lines[i][0], lines[i][1], color, 1, cv.LINE_AA)
```

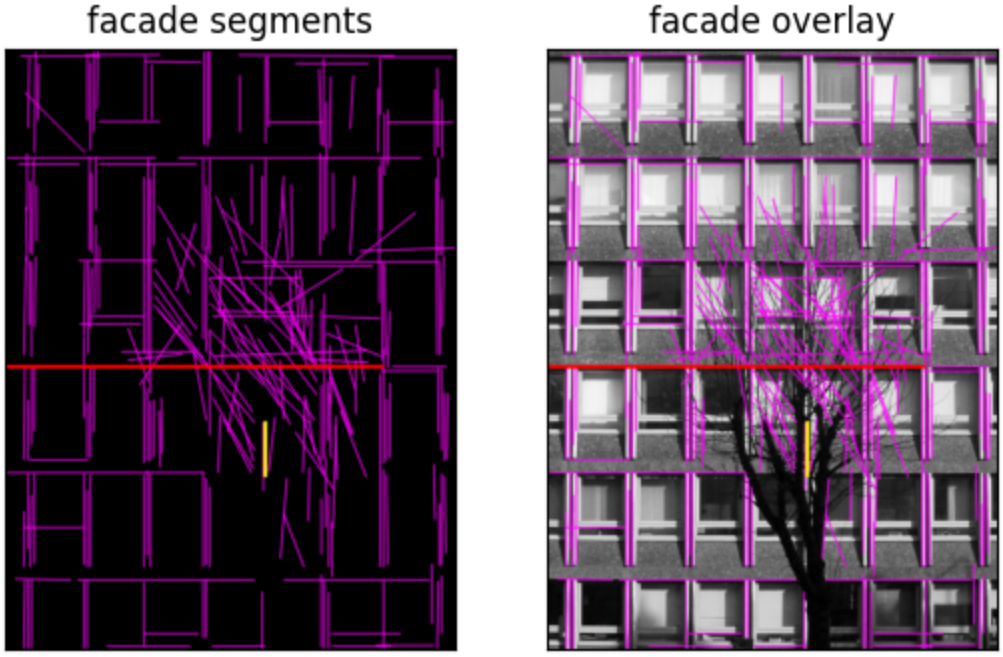
1.2.4 Self-work

> ***Self-work***
>
> Take **three** arbitrary images and execute the probabilistic Hough transform with the scikit-image library. Display the found lines, print their number, and print the shortest and the longest line parameters.
>
> ***Notes.***
>
> 1. *You may have to change the Hough transform parameters to match your images.*

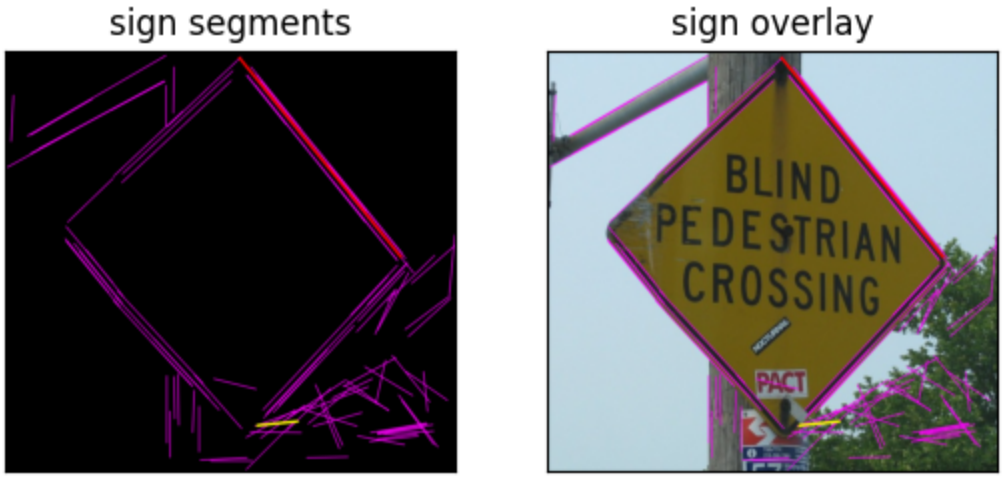
In [14]:
```
for name, img in line_sources.items():
  edges = cv.Canny(img, 60, 180)
  segments = skimage.transform.probabilistic_hough_line(edges, threshold = 4

  canvas = img.copy()
```

```
  blank = np.zeros_like(img)
  if segments:
    lengths = [math.hypot(b[0] - a[0], b[1] - a[1]) for a, b in segments]
    short, long = int(np.argmin(lengths)), int(np.argmax(lengths))
    for a, b in segments:
      cv.line(canvas, a, b, (255, 0, 255), 1, cv.LINE_AA)
      cv.line(blank, a, b, (255, 0, 255), 1, cv.LINE_AA)
    for layer in (canvas, blank):
      cv.line(layer, segments[short][0], segments[short][1], (0, 255, 255),
      cv.line(layer, segments[long][0], segments[long][1], (0, 0, 255), 2, c

    print(f"{name}: {len(segments)} segments")
    print(f"  shortest {lengths[short]:.1f}px {segments[short][0]}-{segments
    print(f"  longest {lengths[long]:.1f}px {segments[long][0]}-{segments[lo
  ShowImages([(f"{name} segments", blank), (f"{name} overlay", canvas)], 2)
```

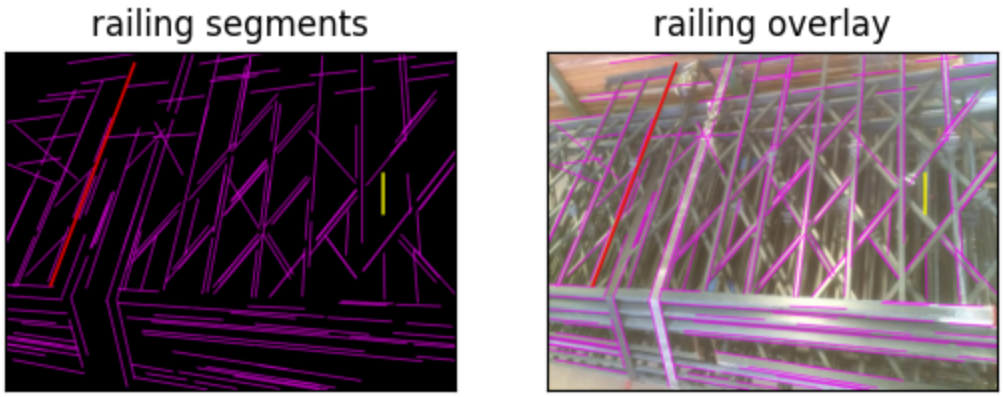
```
facade: 256 segments
  shortest 50.0px (242, 398)-(242, 348)
  longest 351.0px (351, 296)-(0, 296)
```




```
sign: 98 segments
  shortest 50.2px (312, 464)-(362, 459)
  longest 317.8px (490, 255)-(290, 8)
```




```
railing: 186 segments
  shortest 50.0px (468, 202)–(468, 152)
  longest 294.0px (56, 291)–(160, 16)
```



Task 2. Search for Circles

Take three arbitrary images containing circles. Search for circles of a known radius range using the Hough transform. Plot the found circles on the original image and print the number of found circles.

When searching for circles, the Hough parameter space becomes 3-dimensional. The third dimension here is added for the circle radius. Since the circle radius can be of an infinite range, it is required to define this range when executing the Hough transform operations to create the correct accumulator space 3D matrix.

2.1 Hough Transform for Circles with OpenCV

The search for circles with OpenCV is rather straightforward. You don't need to do any image preprocessing as the Canny algorithm is built into a Hough transform implementation. To run Hough transform for circles with OpenCV you have to use the following function:

- `cv.HoughCircles(image, method, dp, minDist, param1, param2, minRadius, maxRadius) -> circles` function is used to execute the Hough transform for Circles on a **grayscale** `image`. It returns a 2-dimensional array of circles where each circle has three parameters: 0 0 - for the center of the circle and its radius. The Hough transform for circles is parameterized in the following way:
  - `method` parameter defines one of two possible methods for this transformation, it can be either `cv2.HOUGH_GRADIENT` or `cv2.HOUGH_GRADIENT_ALT`. These methods are almost similar, however the second one should give a bit better accuracy.
  - `dp` parameter defines the scale ratio of the accumulator matrix. The resolution of the accumulator matrix is `dp` times smaller than the resolution of the source image. In most cases, would fit well.
  - `minDist` parameter defines the minimum distance between circle centers to be detected.
  - `param1` is a high threshold that is passed to the Canny edge detector, while the low threshold would be equal to half of `param1`.
  - `param2` parameter defines the threshold for the circle detection. In the case of the `cv2.HOUGH_GRADIENT` method, it is the accumulator matrix value, while for the `cv2.HOUGH_GRADIENT_ALT` method it is the circle quality measure defined in range where is a perfect circle.
  - `minRadius` and `maxRadius` parameters define the circle radius search range. This will define the third dimension of the Hough parameter space accumulator matrix range.

To display the found circles the `cv2.circle(image, center, radius, color, thickness, lineType)` function may be used to draw a circle on the `image` with the given `center` point, `radius`, and `thickness`. The parameter is used to define the type of the line, which can be set to `cv2.LINE_AA` for an antialiased line.

For example, drawing a circle on an image `Iout` with center and radius and thickness with blue color would be written as follows:

```
cv2.circle(Iout, (50, 50), 10, (255, 0, 0), 1)
```

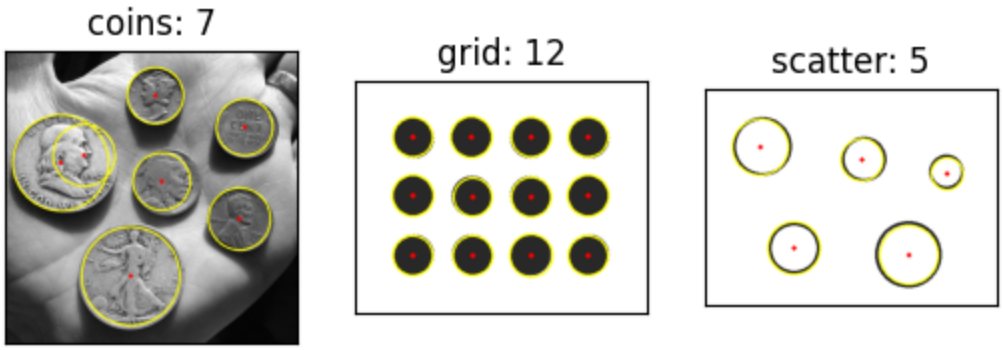
2.1.1 Self-work

> ***Self-work***
>
> Implement the search for circles with the Hough transform and OpenCV.
>
> Take **three** arbitrary images and execute the Hough transform for circles with OpenCV. Display the found circles, and print their number.

> ***Note.*** *You may have to change the Hough transform parameters to match your images.*

In [15]:
```
def grid_circles(rows, cols, step, r):
    h, w = rows * step + step, cols * step + step
    img = np.full((h, w, 3), 255, np.uint8)
    for i in range(rows):
        for j in range(cols):
            cv.circle(img, (step + j * step, step + i * step), r, (40, 40, 4
    return img

coins_gray = cv.imread("images/coins.jpg", cv.IMREAD_GRAYSCALE)
coins_bgr = cv.cvtColor(coins_gray, cv.COLOR_GRAY2BGR)
grid = grid_circles(3, 4, 80, 28)
scatter = np.full((340, 460, 3), 255, np.uint8)
for c, r in [((90, 90), 44), ((250, 110), 33), ((380, 130), 25), ((140, 250)
    cv.circle(scatter, c, r, (35, 35, 35), 3)

pictures = [("coins", coins_bgr, dict(dp=1.2, minDist=40, param1=120, param2
            ("grid", grid, dict(dp=1.2, minDist=50, param1=150, param2=25, m
            ("scatter", scatter, dict(dp=1.2, minDist=50, param1=150, param2

panel = []
for name, img, p, blur in pictures:
    g = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
    g = cv.GaussianBlur(g, (5, 5), 1) if blur == "gauss" else cv.medianBlur(
    found = cv.HoughCircles(g, cv.HOUGH_GRADIENT, p["dp"], p["minDist"], par
    canvas = img.copy()
    count = 0
    if found is not None:
        count = found.shape[1]
        for x, y, r in np.uint16(np.around(found))[0]:
            cv.circle(canvas, (x, y), r, (0, 255, 255), 2)
            cv.circle(canvas, (x, y), 2, (0, 0, 255), 3)
    print(f"{name}: {count} circles")
    panel.append((f"{name}: {count}", canvas))
ShowImages(panel, 3)
```

```
coins: 7 circles
grid: 12 circles
scatter: 5 circles
```



2.2 Hough Transform for Circles with scikit-image

When implementing the Hough transform for circles with scikit-image library you have to do the same two steps as you did for lines. These are:

1. Calculation of the Hough parameter space;
2. Search for peaks in the Hough parameter space to find circles.

Since the Hough transform is split into two steps it means that with scikit-image we can check the Hough parameter space as well.

The following functions for Hough transform for lines are provided in scikit-image:

- `skimage.transform.hough_circle(image, radius, normalize, full_output) -> hspaces` function performs the Hough circle transform of an `image` and calculates the 3-dimensional Hough accumulator matrix. The `radius` parameter is either a scalar to search for a single radius or an array of radii. Two optional parameters allow to `normalize` the output accumulator spaces by the number of pixels required to draw a circle to make different radii circles to have the same weight in the accumulator (*True* by default) and to increase the size of the accumulator by two maximum radii to be able to get `full_output` and find circles with centers at outside of the image (*False* by default). It returns a 3D accumulator Hough parameter space matrix (an array of 2D Hough parameter spaces for each of the radii from the `radius` list).
- `skimage.transform.hough_circle_peaks(hspaces, radii, min_xdistance, min_ydistance, threshold, num_peaks, total_num_peaks, normalize) -> accum, cx, cy, rad` function performs the search for peak values in the 3D Hough parameter space `hspaces` acquired at the previous step with an array of `radii`. Optional parameters allow specifying the minimum distances between picked circle centers (`min_xdistance` along and `min_ydistance` along), the `threshold` parameter for peaks (half of the maximum `hspaces` value by default), the maximum number of peaks selected in a single layer Hough parameter space and all spaces total (`num_peaks` and `total_num_peaks` parameters correspondingly). Also, it's possible to `normalize` peaks by the radius as otherwise the peaks of the higher radius would be preferred over the the smaller radius (default behavior).

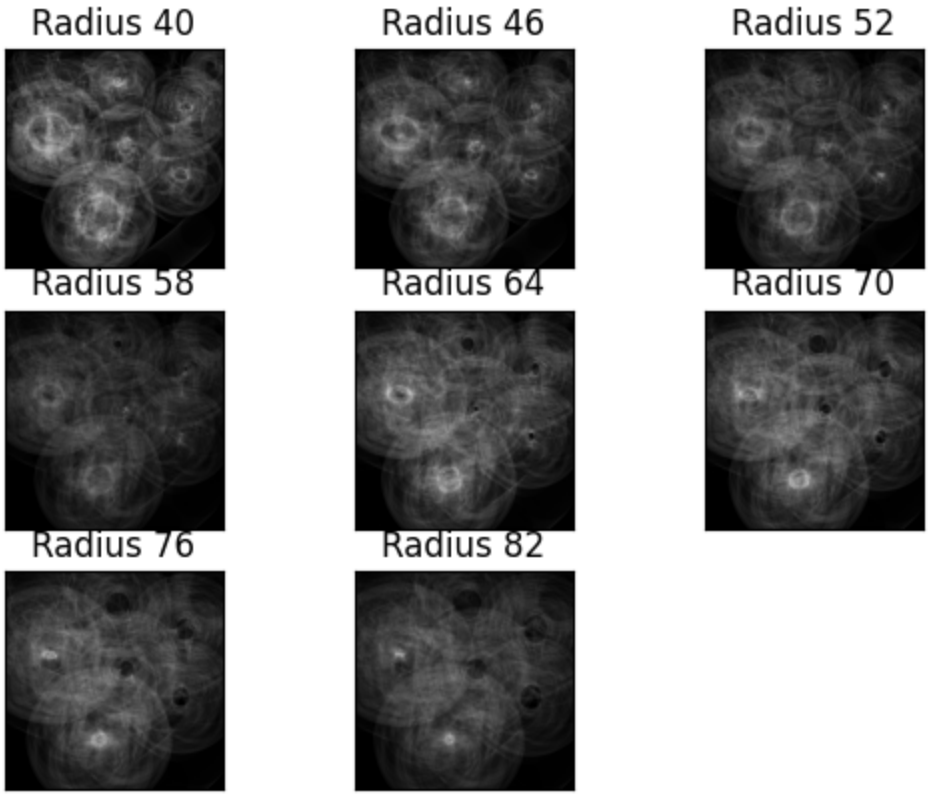
It should be noted that scikit-image also provides a function for the Hough transform for ellipses which is out of the current practical assignment scope. If you are interested, you may try it as well. The arguments are similar to the one used for circles, however, it also allows searching for ellipses with the specified parameters range and returns an array of tuples with found ellipses: `skimage.transform.hough_ellipse(image, threshold = 4, accuracy = 1, min_size = 4, max_size = None) -> [(accumulator, yc, xc, a, b, orientation)]`

Let us take an image with coins and try finding them by the Hough transform. Since scikit-image libraries work with black-and-white images, we have to preprocess it with Canny algorithm to find edges first.

In [16]:
```
# Read an image from file in BGR
fn = "images/coins.jpg"
I2 = cv.imread(fn, cv.IMREAD_COLOR)
if not isinstance(I2, np.ndarray) or I2.data == None:
  print("Error reading file \"{}\"".format(fn))
  exit()

# Preprocess with Canny algorithm
I2edge = cv.Canny(I2, 200, 250)
# Display it
ShowImages([("Source image", I2),
            ("Edges", I2edge)], 2)
```

Now we can run the Hough circles transform. Since we know that we are searching for circles in the range from to pixels, so we will define it as our radii range with step .

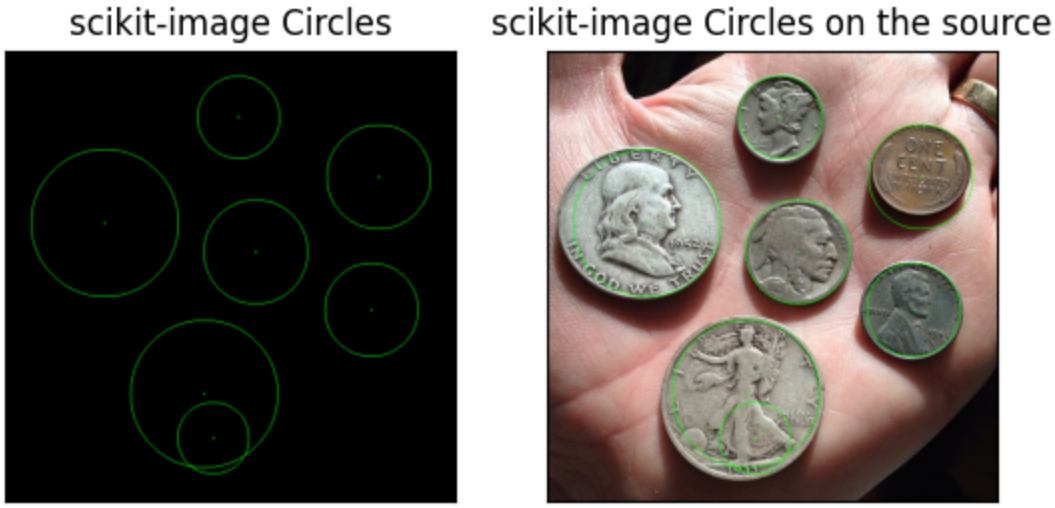
In [17]:
```
# Find circles with Hough transform
# Detect circles with radii in range from 40 to 85 with step 6
radii = np.arange(40, 86, 6)
I2h = skimage.transform.hough_circle(I2edge, radii)

# Show Hough parameter space
# We will resize it to make square for a better display
spaces = []
for i in range(len(radii)):
  spaces.append(("Radius {}".format(radii[i]), cv.resize(I2h[i].astype(np.fl
ShowImages(spaces, 3)
```



Now we can search for maxima values in the Hough parameter spaces and draw the found circles with subsequent calls to the `cv2.circle()` function.

In [18]:
```
# Select the most prominent 10 circles
# Minimum distance between centers along X: 30
# Minimum distance between centers along Y: 30
I2accum, I2cx, I2cy, I2radii = skimage.transform.hough_circle_peaks(I2h, rad

# Create output images
I2out = I2.copy()
I2circles = np.zeros_like(I2)

# Go through all found circles and display them
if I2cx is not None and I2cy is not None and I2radii is not None:
  for i in range(0, len(I2cx)):
    center = np.int32((I2cx[i], I2cy[i]))
    radius = int(I2radii[i])
    cv.circle(I2out, center, 1, (0, 255, 0), 1)
    cv.circle(I2out, center, radius, (0, 255, 0), 1)
    cv.circle(I2circles, center, 1, (0, 255, 0), 1)
    cv.circle(I2circles, center, radius, (0, 255, 0), 1)

# Display it
print("Found {} circles".format(len(I2cx)))
ShowImages([("scikit-image Circles", I2circles),
            ("scikit-image Circles on the source", I2out)], 2)
```

```
Found 7 circles
```

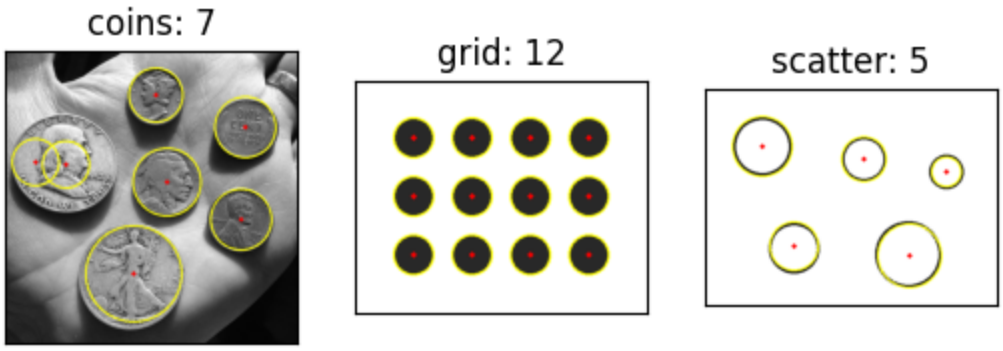


2.2.1 Self-work

> ***Self-work***
>
> Take **three** arbitrary images and execute the Hough transform for circles with the scikit-image library. Display the found circles and print their number.
>
> ***Notes.***
>
> 1. *You may have to change the Hough transform parameters to match your images.*

In [19]:
```
pictures = [("coins", coins_gray, np.arange(40, 86, 6), 40, 7),
            ("grid", cv.cvtColor(grid, cv.COLOR_BGR2GRAY), np.arange(18, 42,
            ("scatter", cv.cvtColor(scatter, cv.COLOR_BGR2GRAY), np.arange(2

panel = []
for name, g, radii, mind, npeaks in pictures:
    edges = cv.Canny(g, 80, 160)
    spaces = skimage.transform.hough_circle(edges, radii)
    _, cx, cy, cr = skimage.transform.hough_circle_peaks(spaces, radii, min_
    canvas = cv.cvtColor(g, cv.COLOR_GRAY2BGR)
    for x, y, r in zip(cx, cy, cr):
        cv.circle(canvas, (int(x), int(y)), int(r), (0, 255, 255), 2)
        cv.circle(canvas, (int(x), int(y)), 2, (0, 0, 255), 3)
    print(f"{name}: {len(cx)} circles")
    panel.append((f"{name}: {len(cx)}", canvas))
ShowImages(panel, 3)
```

```
coins: 7 circles
grid: 12 circles
scatter: 5 circles
```



Task 3. Optional

Implement the classic Hough transform algorithms for lines by yourself. Compare your implementation results with the ones obtained in the first parts of the assignment. Highlight the selected points in the Hough parameter space.

To implement the Hough transform for straight lines you have to allocate an accumulator matrix manually and implement the classic algorithm for the Hough transform to fill the accumulator matrix:

```
For each point of the source space (x,y)
  For theta = 0 to 180
    rho = x cos theta + y sin theta
    H(theta, rho) = H(theta, rho) + 1
  end
end
```

After the accumulator matrix is filled you should select maxima. The simplest way to do it is to use the thresholding method you learned in the previous class.

Notes.

1. *Python implementation for this algorithm would be very slow so consider taking the low-resolution input image.*
2. *Don't forget to run the Canny algorithm before the Hough transform to filter the edge points.*

> ***Self-work (optional)***
>
> Implement the Hough transform for straight lines. Display the result.

In [20]:
```
# TODO Place your solution here
```

Questions

Please answer the following questions:

- *What is the main principle of the Hough transform?*

> Суть — переход из пространства картинки в пространство параметров фигуры. Через краевую точку проходит целое семейство одинаковых фигур, и точка добавляет голос каждой. Где семейства многих точек пересекаются — там реальная прямая или окружность, и ячейка набирает максимум. Поэтому метод терпим к шуму и разрывам контура.

- *May the Hough transform be used to find arbitrary contours that cannot be described analytically?*

> Можно, через обобщённый вариант (Generalized Hough Transform, Ballard, 1981). Раз формулы нет, эталон описывают R-таблицей смещений до опорной точки по направлению градиента. На новом изображении точки голосуют за позицию этой точки, пик показывает, где объект. Добавив масштаб и угол, ищут форму в разных видах.

- *What are the recurrent and generalized Hough transforms?*

> Generalized — обобщение на нестандартные формы через R-таблицу. Recurrent (рекуррентное) — последовательное сокращение размерности: параметры находят не все сразу, а по группам, закрепляя уже найденное, что экономит память аккумулятора и ускоряет счёт.

- *What are the other ways of line parametrization in the Hough transform?*

> Помимо (rho, theta) встречается наклон-сдвиг y = kx + b, но он плох вертикалями, где k стремится к бесконечности. Поэтому на практике берут нормальную форму с конечными rho и theta. Ещё прямую задают парой точек на краях изображения — это удобно для вероятностного варианта, который и так возвращает отрезки.

Conclusion

What have you learned with this task? Don't forget to conclude it.

> Преобразование Хафа щупала на разных снимках: для линий — фасад здания, дорожный знак и металлическая решётка, для окружностей — монеты плюс два синтетических изображения (сетка кругов и разбросанные кольца), итого три картинки на каждую задачу с окружностями.

> Несколько наблюдений по ходу. Метод целиком зависит от карты границ — порог Canny меняет, сколько примитивов проголосует. Классический HoughLines берёт там, где нужны направления линий целиком, вероятностный — где важны конкретные отрезки и их длины. scikit-image открывает доступ к аккумулятору, и на регулярных текстурах прямо видно полосы-максимумы, по которым удобно ставить порог пиков.
>
> Для окружностей в OpenCV всё решали param2 и диапазон радиусов, в scikit-image — шаг сетки радиусов и минимальные расстояния между центрами. Если коротко: Хаф устойчив, но требует ручной настройки под каждую сцену.